Research Proposal

Jonathan Reicher Shachar Itzhaki

May 25, 2026

Problem Statement

In the intersection between the worlds of mathematical theory, formal logic, and programming, exists a species of tools called proof assistants. These are programming languages, meant not for building software, but for building mathematical proofs. Proof assistants, contrary to pen and paper, check formalized proofs for correctness and soundness. These systems have existed for many years, but recently, one in particular has been gaining traction.

Lean (TODO: cite?) is an interactive proof assistant geared for a general math audience. Two things that make it especially appealing are its large ecosystem and its intuitive syntax. Despite that, there are some challenging aspects of adopting Lean, and one in particular is how complex its semantics and type system are, and how.

We propose building a system that will reduce the friction of using Lean by being more "forgiving" and less formal. The system will allow users to write down proofs in way that humans can read and understand as written, with parts of the proof being translated to Lean, to help a user start formalizing their proof in Lean. This workflow is in accordance to how researchers tend to use Lean, as a formal proof in Lean often comes as a supplementary part of a paper proof inside a research paper.

Existing Solutions

For starting a project in Lean, one can reach for any off-the-shelf generative large language model such as ChatGPT. These systems have gotten good at generating good and correct Lean code. One can write their paper, give it to the system, and prompt it to generate a formal Lean proof. There are even tools in place to help large language models

Proposition	Explanation
$\lvert D\rvert = 50$	Our board's size is 100, and every element is a set of size 2, hence: 50 tiles
$(D \cap H) \cup (D \cap V) = D$	By $D \subseteq H \cup V$
$(D \cap H) \cap (D \cap V) = \varnothing$	Because H and V are disjoint!
$\lvert D \cap H\rvert = \lvert D \cap V\rvert = 25$	By the assumption, and by previous two conjectures
$\sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap V\}\!\}$ is even	Every number in the sum appears twice, as the x's of a tile's cells are equal!
$\sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap H\}\!\}$ is odd	Here every number appears with its successor by similar reasoning
$\sum\{\!\{x \mid \langle x, y\rangle \in 0..9 \times 0..9\}\!\}$ is even	By decide
$\sum\{\!\{x \mid \langle x, y\rangle \in 0..9 \times 0..9\}\!\} = \sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap V\}\!\} + \sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap H\}\!\}$	todo
Contradiction	todo

Figure 1: A proof sketch of the problem.

work with Lean such Aristotle. These systems suffer a number of drawbacks: They lack reproducibility, are usually based on cloud services, and are not fit with an interactive and iterative workflow.

There are also a number of automated proof machinery that can be embedded into Lean, such as Aesop and Grind. These systems work on top of Lean by generating parts of a proof that you request. This is useful as part of interactive theorem proving, but they do not produce a readable proof. They can work off of a paper, but that requires manual work.

TODO: Mention open ai's paper that adi sent

Preliminary Work

We started from taking a look at a case study example of a proof in Lean.

> Can you cover a 10×10 board with 1×2 tiles, such that the number of horizontal and vertical tiles is the same?

As it turns out, you can't. One can prove that it is impossible. A short intuitive proof can be seen in Figure 1. While this proof is readable and correct, and even relatively formal, it does not translate well to Lean.

As an example, here is a part of the proof that represents conjecture ???.

One of the challenges in translating a proof to Lean is representation. It is often much more idiomatic to use one of many possible representations. For something that might fit as a set for a mathematician, in Lean it might be better use the `Set` type, or the `Finset`, or maybe represent it as a type, or even a predicate. The most idiomatic choice for representation depends on use.

As a proof of concept, we made a tool which compiles inline math in a markdown document to Lean code, and uses a static analysis of the document to determine which representation to use for each mathematical object. The tool we have right now is very basic, and the only possible representations are `Set` or `Finset` for sets, and `Multiset` for multisets and `Prod` for tuples.

References